Conflict-Graph Construction: the Four Algorithms of the Main Suite

Ido Elbak

October 2026

Problem and notation

A block is a sequence of transactions $T_0, \ldots, T_{n-1}$. Transaction t has a write set $W(t)$ and a read set $R(t)$ over objects with dense integer IDs. The conflict graph has an edge $\{i, j\}$ iff $W(i) \cap W(j) \neq \emptyset$, $W(i) \cap R(j) \neq \emptyset$ or $R(i) \cap W(j) \neq \emptyset$; read-read sharing is no edge. Every algorithm's output is checked bit for bit against the all-pairs reference on every block of every run.

A *row* is a bitset over the block's transactions, $\lceil n/64 \rceil$ 64-bit words wide; $\mathbf{1}_t$ is the bitset holding only t. An object is *shared* when it is written and a second transaction touches it. Only shared objects can produce an edge: a read of a never-written object conflicts with nobody, and neither does a written object no other transaction touches. Every algorithm applies this filter, allocates nothing per call (all scratch is reused) and sizes its rows to the live block. The parallel paths use one shared worker pool (ParallelFor: workers claim chunks through one atomic counter).

Algorithm	Output	Idea
BitmapTiny	full $n \times n$ matrix	the baseline: per-object bitsets, ORed word-wide into each row
HybridJoinTiny	full $n \times n$ matrix	the strongest competitor: sort-based join, per object the cheaper of pair bits and row ORs
IslandHub	island graphs + hub matrix	ours: Union-Find islands without the hub objects; hubs as a bit-matrix over their touchers
IslandBipartite	island graphs + hub lists	ours: as IslandHub, hubs as per-hub writer and reader lists

1 The one-word kernel (shared by all four)

For blocks of at most 64 transactions every row is one word. There the shared-object filter costs more than the ORs it saves, and it is not needed for correctness: a private object's bits are the transaction's own (cleared with the self bit), and a never-written object has no writer bits.

Algorithm 1 TinyBitmap($T_0..T_{n-1}$), $n \leq 64$

Input: per object o: one-word bitsets $w[o], r[o]$, all zero

for $t \leftarrow 0$ **to** $n-1$ **do**
  for $o \in W(t)$ **do** $w[o] \leftarrow w[o] \vee \mathbf{1}_t$
  for $o \in R(t)$ **do** $r[o] \leftarrow r[o] \vee \mathbf{1}_t$
for $t \leftarrow 0$ **to** $n-1$ **do**
  $row_t \leftarrow \bigvee_{o \in W(t)} (w[o] \vee r[o]) \;\vee\; \bigvee_{o \in R(t)} w[o]$
  $row_t \leftarrow row_t \wedge \neg \mathbf{1}_t$
zero w, r (word-wide if objects $\leq 16n$, else access by access)

2 BitmapTiny (the baseline)

Algorithm 2 BitmapTiny(T)

if $n \le 64$ **then return** TinyBitmap(T)
Index: per object, writer and reader bitsets $W[o], R[o]$ of $\lceil n/64 \rceil$ words, and a state
for t, $o \in W(t)$ **do**
    $state[o] \leftarrow t+1$ if $state[o] = 0$, else mark o shared; $W[o] \leftarrow W[o] \vee \mathbf{1}_t$
for t, $o \in R(t)$ with $state[o] \neq 0$ **do**
    mark o shared; $R[o] \leftarrow R[o] \vee \mathbf{1}_t$
Sweep, straight into the output rows (inline up to 518 transactions, else ParallelFor, 16 rows per chunk)
for $t \leftarrow 0$ **to** $n-1$ **do**
    $row_t \leftarrow \bigvee_{o \in W(t),\ o \text{ shared}} (W[o] \vee R[o]) \ \vee \bigvee_{o \in R(t),\ o \text{ shared}} W[o]$
    clear bit t of row_t
zero exactly the words and states the index set ▷ O(accesses)

Both triangles are produced directly and every row is overwritten, so the output needs no clearing. The cost is one $\lceil n/64 \rceil$-word OR per shared access.

3 HybridJoinTiny (the strongest competitor)

The database sort-based conflict join, with SpGEMM's choice of a sparse or a dense accumulator made per object. Sequential.

Algorithm 3 HybridJoinTiny(T)

if $n \le 64$ **then return** TinyBitmap(T)
if $\lceil n/64 \rceil < 4$ **then return** Bitmap(T) ▷ BitmapTiny's path for $n > 64$
group the accesses by object (counting sort): writers $w(o)$, then readers $r(o)$ of written objects, each in ascending transaction order
clear the output matrix
for each object o with $|w(o)| \ge 1$ and $|w(o)| + |r(o)| \ge 2$ **do**
    $pairs \leftarrow |w|(|w|-1) + 2|w||r|$
    if $pairs \le (|w| + |r|) \cdot \lceil n/64 \rceil$ **then** ▷ cold object
        for every writer pair and writer–reader pair $\{a, b\}$: set bit b of row_a and bit a of row_b
    else ▷ hot object
        $A \leftarrow W \leftarrow \bigvee_{a \in w} \mathbf{1}_a$; $A \leftarrow A \vee \bigvee_{b \in r} \mathbf{1}_b$ ▷ two scratch rows, L1-resident
        for $a \in w$ **do** $row_a \leftarrow row_a \vee A$; clear bit a
        for $b \in r$ **do** $row_b \leftarrow row_b \vee W$
        zero W, A

Per object the cost is min(pair bits, accessors$\cdot\lceil n/64 \rceil$): cold objects cost their pairs, hot objects are swept word-wide as in Bitmap but from two scratch rows instead of an index of objects $\times \lceil n/64 \rceil$ words.

4 IslandHub and IslandBipartite (the Union-Find family)

One algorithm with two hub layers; the knobs and everything else are identical.

Output (`model.IslandTopology`). The *islands*: groups of at least two transactions, each with its own symmetric conflict matrix in local coordinates (k members, $\lceil k/64 \rceil$-word rows). Islands do not conflict with each other except through hub objects, so they can be colored in parallel. The *hub layer* holds the conflicts through hub objects: IslandHub as a bit-matrix over the transactions touching a hub; IslandBipartite as, per hub, its writer list and its reader list (each writer conflicts with every other writer and every reader of the hub). The union of the two parts is the conflict graph. Transactions in neither part have no conflicts.

The partition. Per object an *anchor*: 0 if not written yet, else (root of its first writer) + 1, a *shared* flag once a second transaction touches it, or a hub number. The union-find forest uses union by size, path halving, and a circular *next* list per set, so the members of an island are read off without a Find per transaction and without a sort.

Algorithm 4 Partition(T, hubs)

for $t \leftarrow 0$ **to** $n-1$ **do** ▷ writer pass
  $root \leftarrow t$
  for $o \in W(t)$ **do**
    if $anchor[o] = 0$ **then** $anchor[o] \leftarrow root + 1$ ▷ first writer
    else if o is a hub **then** record (t, o) as a hub write ▷ unions nothing
    else mark o shared; $root \leftarrow$ Union($root$, Find($anchor[o] - 1$))
for $t \leftarrow 0$ **to** $n-1$ **do** ▷ reader pass
  for $o \in R(t)$ with $anchor[o] \neq 0$ **do** ▷ never-written objects are skipped
    if o is a hub **then** record (t, o) as a hub read
    else mark o shared; Union(Find(t), Find($anchor[o] - 1$)) ▷ t's root looked up once

Algorithm 5 IslandHub(T) / IslandBipartite(T)

if $n \leq 64$ **and** $\sum_t(|W(t)| + |R(t)|) - \#\text{objects} \geq \lceil 0.5\,n \rceil$ **then**
  return TinyBitmap(T) as one island ▷ tiny block with repeated objects
hubs $\leftarrow \emptyset$
if $n \geq 64$ **then**
  hubs $\leftarrow$ objects written by ≥ 3 of $T_0..T_{63}$ ▷ prefix hub rule
  if $n \geq 512$ **and** ≥ 32 of $T_0..T_{63}$ write a hub **then**
    return HybridJoin(T) as one island ▷ dense block: one island would hold most of it
Partition(T, hubs); collect the islands (sets of ≥ 2 members)
for each island I, $k = |I|$ **do** ▷ inline, or one ParallelFor pass when the work is large
  if $k = 2$ **then** one edge ▷ the union implies a direct conflict
  else if $k \leq 4$ **then** check every pair directly
  else
    compact the members' accesses of shared non-hub objects (branch-free)
    if $k \leq 64$ **then** the one-word kernel on them ▷ TinyBitmap in local coordinates
    else if $k \geq 0.3\,n$ **then** HybridJoin's per-object kernel on them ▷ giant island
    else BitmapTiny's index, sweep and wipe with $\lceil k/64 \rceil$-word rows
build the hub layer (Algorithm 6)

Algorithm 6 The two hub layers

IslandHub (bit-matrix): number the touchers q in first-hub-access order
per hub h: bitsets W_h, R_h over the touchers
if some hub is written by every toucher **then** ▷ e.g. Ethereum's fee recipient
  every row $\leftarrow$ all ones except the own bit
else
  for each toucher q **do** $row_q \leftarrow \bigvee_{h \text{ written by } q}(W_h \vee R_h) \;\vee \bigvee_{h \text{ read by } q} W_h$; clear bit q
IslandBipartite (lists): counting-sort the recorded hub accesses by hub
HubWriters[h] $\leftarrow$ writers of h, HubReaders[h] $\leftarrow$ readers of h, ascending

IslandBipartite pays one list entry per hub access instead of a clique among a hub's touchers, but it changes the output contract of the hub layer: the conflicts through a hub are implied by its lists rather than stored as edges.

Knobs (defaults)

Knob	Value	Meaning
Competitors		
`BitmapSequentialMax`, `BitmapChunkRows`	518, 16	Bitmap inline up to this size; rows per parallel chunk
`HybridJoinConfig`	MinWords 4, DenseFactor 1	narrower rows run Bitmap; the hot-object test
Island algorithms (`HubConfig`)		
`PrefixTxns`, `PrefixMinWrites`	64, 3	hub = written by ≥ 3 of the first 64 transactions
`HubMinTxns`	64	smaller blocks have no hubs
`TinyMax`, `TinyMinRepeats`	64, 0.5	the tiny path
`DenseMinTxns`, `DenseHubShare`	512, 0.5	the dense fallback
`HubRowsPerUnit`	64	hub-matrix rows per parallel unit
Island core (`IslandConfig`)		
`PairwiseMax`	4	islands of 3–4 members are checked pairwise
`GiantShare`, `JoinDense`	0.3, 1	giant islands take HybridJoin's kernel; its hot-object factor
`SequentialMax`, `ParallelWords`	300, 8000	parallel only above 300 transactions and 8000 words of island matrices
`SplitRows`, `GroupTxns`	16, 256	parallel units: rows of a wide island; members of grouped small islands